

PROJECT REPORT

CH iDesign & Renovation

Website Revamp

PHASE 1 — FRONTEND WITH DUMMY DATA

Project	CH iDesign & Renovation — Website Revamp
Phase	Phase 1 — Frontend with Dummy Data
Status	Complete
Date	28 May 2026
Prepared by	Adron Lim
Client	Interior Design / Renovation Studio — Batu Kawan, Penang

1. Executive Summary

Phase 1 delivers the complete frontend of the CH iDesign & Renovation website as a React single-page application, running entirely on dummy data with no backend. The public portfolio site and a fully interactive admin CMS are both in place, styled to a warm, editorial “luxury interior studio” design language with full light/dark mode support.

All Phase 1 roadmap items are complete. The application builds cleanly for production, passes linting, and is structured so the in-memory data layer can be swapped for Firebase (Firestore + Auth) and Cloudinary in Phase 2 without reworking the UI.

2. Scope of Phase 1

In scope (delivered):

- Public portfolio website — all sections per the brief
- Filterable project gallery with image lightbox
- Light / dark theme with persistence
- Admin CMS — dashboard, project CRUD, upload, settings
- Demo authentication gate on admin routes
- Global in-memory state shaped like a future API client

Explicitly out of scope (deferred to Phase 2):

- Real backend / database (Firestore)
- Real authentication (Firebase Auth)
- Real image uploads / CDN (Cloudinary)
- Deployment (Firebase Hosting)

3. Tech Stack

TOOL	VERSION	PURPOSE
Node.js	22 LTS	Runtime (pinned via <code>.nvmrc</code>)
Vite	8.x	Build tool & dev server
React	19.x	UI framework (function components + hooks)
react-router-dom	7.x	Client-side routing
Tailwind CSS	v4 (<code>@tailwindcss/vite</code>)	Utility-first styling (config-less)
Yarn	1.22.x	Package manager

4. Architecture & Project Structure

```
src/
  App.jsx           providers + routes
  main.jsx          entry
  index.css         Tailwind import + @theme tokens + dark-mode variant
  context/
    ThemeContext.jsx light/dark theme (data-theme attr, localStorage)
    ProjectsContext.jsx in-memory projects + settings store, CRUD, demo auth
  components/
    Navbar.jsx      logo, nav, theme toggle, CTA, mobile drawer
    Hero.jsx        split layout with stat cards
    About.jsx       intro + 3 service pillars
    ProjectGrid.jsx filterable masonry grid
    ProjectCard.jsx individual project tile
    ProjectLightbox.jsx gallery modal
    Process.jsx     4-step process
    Testimonials.jsx client reviews (dark band)
    ContactForm.jsx enquiry form + company info
    Footer.jsx      brand / nav / services / contact columns
  pages/
    Home.jsx        public portfolio (composes the sections)
  admin/
    Login.jsx       demo sign-in
    RequireAuth.jsx route guard
    AdminLayout.jsx sidebar shell
    Dashboard.jsx   KPI widgets + recent projects
    ProjectsTable.jsx full CRUD table + edit modal
    UploadProject.jsx form with drag-and-drop image upload
    Settings.jsx    edit company information
  data/
    projects.js     dummy project data + category list
    settings.js     company settings, demo credentials, testimonials,
                   services, process steps
  docs/
    phase-1-frontend.md this document (source)
    phase-1-frontend.pdf this document (deliverable)
```

5. Routing

ROUTE	COMPONENT	NOTES
/	Home	Public portfolio
/admin/login	Login	Public sign-in
/admin	AdminLayout (guarded)	Index → Dashboard
/admin/projects	ProjectsTable	Full CRUD
/admin/upload	UploadProject	Add project
/admin/settings	Settings	Edit company info

The `/admin` tree is wrapped in `RequireAuth`. This is a **Phase 1 demo gate**, not real security — see §10.

6. Features Delivered

6.1 Public Website

- **Navigation** — sticky bar with logo, anchor links, dark/light toggle, “Get a quote” CTA, admin link, and a mobile drawer.
- **Hero** — editorial headline + dual CTAs on the left; feature image with floating stat cards on the right.
- **About** — studio intro pulled from settings, plus three service pillars (Interior Design, Renovation, Styling & Furnishing).
- **Projects** — filterable masonry grid (All / Living Room / Kitchen / Bedroom / Commercial). Only `published` projects appear. Clicking a card opens a lightbox with the full gallery, description, area, and year.
- **Process** — four-step path: Discovery → Concept → Refinement → Build.
- **Testimonials** — three client quotes on a dark editorial band.
- **Contact** — company info block (phone, email, address, hours) beside an enquiry form with a success state.
- **Footer** — brand, navigation, services, and contact columns.

6.2 Admin CMS

- **Dashboard** — KPI cards (total / published / drafts) and a recent-projects table.
- **Projects** — full table with cover thumbnails, inline publish/draft toggle, an edit modal, and delete-with-confirmation.
- **Upload Project** — form with drag-and-drop image upload (with previews and cover selection), category, area, year, description, and a publish toggle.
- **Settings** — edit company name, tagline, phone, email, address, hours, and about text; changes propagate live across the public site.

All admin mutations flow through `ProjectsContext` and reflect immediately in the public site within the same session.

7. Design System

TOKEN	VALUE
Display font	Cormorant Garamond (serif, editorial)
Body font	DM Sans (clean sans-serif)
Surfaces	Warm cream <code>#f8f5f0</code> / stone <code>#ede7dd</code>
Accent	Gold <code>#8b6914</code> (light) / <code>#c8952a</code> (dark)
Dark surfaces	Charcoal <code>#2c2417</code> / ink <code>#1c1810</code>

Brand tokens are declared in an `@theme` block in `src/index.css` and consumed as ordinary Tailwind utilities (`bg-cream`, `text-gold`, `font-display`, ...). Dark mode is a class-based `dark:` variant keyed to `data-theme="dark"` on the `<html>` element.

8. State Management

Two React contexts hold all global state in Phase 1:

- **ThemeContext** — exposes `theme` and `toggleTheme`; writes `data-theme` to `<html>`, persists the choice to `localStorage`, and defaults to the OS `prefers-color-scheme`.
- **ProjectsContext** — seeded from `src/data/`. Exposes `projects`, `published` (status-filtered), `settings`, and `isAuthenticated`, plus the mutators `addProject`, `updateProject`, `deleteProject`, `updateSettings`, `login`, and `logout`. The CRUD surface is deliberately shaped like a future API / Firestore client so Phase 2 can swap the internals without touching the components.

9. Data Model

```
// src/data/projects.js
{
  id: 1, // number (Firestore: auto doc id)
  name: 'The Serenity Residence',
  cat: 'living', // 'living' | 'kitchen' | 'bedroom' | 'commercial'
  catLabel: 'Living Room', // display label
  area: '1,800 sqft',
  year: '2024', // string
  status: 'published', // 'published' | 'draft'
  img: 'https://...', // cover image URL
  images: ['https://...'], // gallery URLs
  desc: '...', // description
}
```

```
// src/data/settings.js – company settings
{
  name, tagline, phone, email, address, hours, about
}
```

10. Demo Admin Credentials

FIELD	VALUE
Email	admin@chidr.com.my
Password	password

Credentials are hardcoded in `src/data/settings.js` and the session is stored in `sessionStorage`. This is a demo stand-in only — **not** real authentication. Phase 2 replaces it with Firebase Auth.

11. How to Run

```
nvm use          # Node 22 (from .nvmrc)
yarn install      # install dependencies
yarn dev          # dev server → http://localhost:5173
yarn build        # production build → dist/
yarn preview      # serve the production build
yarn lint         # ESLint over the repo
```

Verification status: `yarn lint` passes with no errors; `yarn build` completes successfully; the dev server boots and serves the app.

12. Known Limitations (Phase 1)

- **State is in-memory.** Admin edits, uploads, and settings changes reset on a full page reload. This is intentional for Phase 1 — Firestore makes them durable in Phase 2.
- **Images are remote (Unsplash).** Project thumbnails require an internet connection to render. Uploaded images use temporary object-URL previews, not real uploads.
- **Auth is cosmetic.** The `RequireAuth` gate keeps casual visitors out of the admin UI but provides no real security.

13. Phase 2 Roadmap (Next Steps)

- Set up Firebase project + Firestore + Auth.
- Replace dummy data with Firestore reads; keep the `ProjectsContext` API.

- Add a real Firebase Auth guard on `/admin` routes.
- Integrate Cloudinary for image uploads in `UploadProject`.
- Deploy to Firebase Hosting.

Firestore cost-trap mitigations (scoped listeners, cursor pagination, counter documents, guarded writes) are documented in `context.md` and must be settled before writing Firestore code.